0.66. La lettre bouge avec la croix

Le joueur agit, l'écran répond : c'est déjà un jeu.

1. Ce qu'il faut comprendre

`bouton(DROITE)` vaut vrai **tant que** la flèche droite est enfoncée. Même chose pour `GAUCHE`, `HAUT` et `BAS`.

Bouger à chaque image ferait traverser l'écran en un tiers de seconde. `attente` impose donc **8 images** entre deux pas : en gardant la flèche enfoncée, la lettre avance d'une case environ sept fois par seconde.

Les conditions `x < 19`, `x > 0`, `y < 17` et `y > 0` empêchent de sortir de l'écran.

Voilà les trois ingrédients d'un jeu : la **boucle**, le **joueur qui agit**, et l'**écran qui répond**.

Les fonctions de cette leçon — et le tuto de chacune

- `#include <effacer>` → 0.16.3. La fonction effacer() — effacer des cases
- `#include <bouton>` → 0.65.5. La fonction bouton() — lire la manette
- `#include <poser>` → 0.1.3. La fonction poser() — poser une tuile
- `#include <ALPHABET>` → 0.1.4. ALPHABET — les lettres de la police

📖 Les mots de cette leçon — si tu débutes

- **accolades { }** — Elles entourent un bloc : les lignes qui vont ensemble (le corps d'une fonction, d'une boucle, d'un « if »). Chaque « { » a son « } ».
- **affectation (=)** — « x = 5; » : la boîte x prend la valeur 5. Ce n'est pas une égalité : c'est un rangement.
- **argument** — Une valeur donnée à une fonction quand on l'appelle, entre ses parenthèses : dans texte(5, 8, "SALUT"), 5, 8 et "SALUT" sont les arguments.
- **bloc** — Un groupe de lignes entre accolades { }, qui s'exécutent ensemble.
- **booléen (vrai / faux)** — Le résultat d'une condition : vrai (1) ou faux (0). « true » veut dire vrai, « false » faux.
- **boucle** — Des instructions qui se répètent : « while » (tant que…) ou « for » (pour un compteur qui va de… à…).
- **boucle du jeu** — « while (true) { image(); … } » : ce qui se répète 60 fois par seconde, sans fin, pendant tout le jeu.
- **bouton()** — Une fonction de la console qui dit si un bouton de la manette est enfoncé : 1 oui, 0 non. Les boutons : A, B, HAUT, BAS, GAUCHE, DROITE, START, SELECT.
- **commentaire (//)** — Tout ce qui suit « // » jusqu'au bout de la ligne : une phrase pour l'humain, que la console ignore.
- **condition** — Une question dont la réponse est vrai ou faux : « score > 10 », « bouton(A) ». On la teste avec « if » ou « while ».
- **coordonnées** — Deux nombres qui disent où : la colonne (vers la droite) puis la ligne (vers le bas), à partir de (0, 0) en haut à gauche.
- **if (si)** — Teste une condition : si elle est vraie, on fait le bloc qui suit.
- **image()** — Attend l'image suivante. La console en montre 60 par seconde : c'est ce qui règle la vitesse d'un jeu.
- **#include** — En haut du programme : « #include <texte> » demande la fonction texte() à la console ; « #include "fichier.cpp" » verse un de tes fichiers à cet endroit.
- **incrémenter (++)** — « x++ » ou « x += 1 » : x augmente de 1. « x-- » : x diminue de 1.
- **indice** — Le numéro d'une case dans un tableau, entre crochets : cases[3] est la quatrième case (on compte à partir de 0).
- **instruction** — Un ordre donné à la console, terminé par un point-virgule : « texte(1, 1, "OK"); ».
- **main** — « int main() { … } » : là où le programme commence. La console exécute d'abord la première ligne de main.
- **octet** — Un nombre de 0 à 255 : la plus petite boîte de la mémoire. Une cartouche en a 32 768.
- **opérateurs de comparaison** — == égal, != différent, < plus petit, > plus grand, <= plus petit ou égal, >= plus grand ou égal. Le résultat est vrai ou faux.
- **opérateurs logiques** — && « et » (les deux doivent être vrais), || « ou » (un seul suffit), ! « pas ».
- **point-virgule (;)** — Il termine une instruction, comme le point termine une phrase.
- **poser()** — Met une tuile dans le décor, sur une case de la grille de 20 × 18 (32 × 32 en tout, avec la partie cachée).
- **uint8_t** — Le genre de nombre de cette console : un entier de 0 à 255 (un octet). Au-delà, il repart à 0.
- **variable** — Une boîte avec un nom, dans la mémoire, qui retient un nombre qu'on peut lire et changer.
- **while (tant que)** — Répète un bloc tant qu'une condition est vraie.

🔍 Le programme, ligne par ligne — chaque ligne dite en français

`#include <effacer>  // efface des cases, ou tout le fond`	Demande à la console « effacer » (ce que c'est : efface des cases, ou tout le fond). Sans cette ligne, le compilateur refuserait de l'employer. (Le commentaire le dit : « efface des cases, ou tout le fond ».)
`#include <bouton>  // lit un bouton de la manette`	Demande à la console « bouton » (ce que c'est : lit un bouton de la manette). Sans cette ligne, le compilateur refuserait de l'employer. (Le commentaire le dit : « lit un bouton de la manette ».)
`#include <poser>  // pose une tuile sur une case du fond`	Demande à la console « poser » (ce que c'est : pose une tuile sur une case du fond). Sans cette ligne, le compilateur refuserait de l'employer. (Le commentaire le dit : « pose une tuile sur une case du fond ».)
`#include <ALPHABET>  // les lettres de la police : ALPHABET[0] est le A`	Demande à la console « ALPHABET » (ce que c'est : les lettres de la police : ALPHABET[0] est le A). Sans cette ligne, le compilateur refuserait de l'employer. (Le commentaire le dit : « les lettres de la police : ALPHABET[0] est le A ».)
`uint8_t x = 9;  // la lettre part du milieu de l'écran`	Une variable : une boîte appelée x, qui retient un nombre de 0 à 255, et qui commence à 9. (Le commentaire le dit : « la lettre part du milieu de l'écran ».)
`uint8_t y = 8;`	Une variable : une boîte appelée y, qui retient un nombre de 0 à 255, et qui commence à 8.
`uint8_t attente = 0;  // les images à attendre avant le prochain pas`	Une variable : une boîte appelée attente, qui retient un nombre de 0 à 255, et qui commence à 0. (Le commentaire le dit : « les images à attendre avant le prochain pas ».)
`int main() {`	Le début du programme : la console commence ici, par la première ligne qui suit.
`while (true) {`	La boucle du jeu : tout ce qui est entre les accolades recommence sans fin, 60 fois par seconde avec image().
`image();`	Attendre l'image suivante : la console en montre 60 par seconde. C'est ce qui règle la vitesse du jeu.
`if (attente > 0) attente--;`	Une décision : si attente est plus grand que 0 alors on fait « attente-- ». Sinon, on passe.
`if (attente == 0) {`	Une décision : si attente est égal à 0, on fait ce qui est entre les accolades. Sinon, on passe.
`effacer(x, y, 1); // efface l'ancienne place`	Appel de la fonction effacer de la console, avec les arguments x, y, 1. Ce qu'elle fait : efface un texte à l'écran : « quoi » est le texte lui-même, ou un nombre de cases. Avec un Mot : effacer(SALUT). (Le commentaire le dit : « efface l'ancienne place ».)
`if (bouton(DROITE) && x < 19) { x++; attente = 8; }`	Une décision : si le bouton DROITE est enfoncé et x est plus petit que 19 alors on fait « x++; attente = 8; } ». Sinon, on passe.
`if (bouton(GAUCHE) && x > 0)  { x--; attente = 8; }`	Une décision : si le bouton GAUCHE est enfoncé et x est plus grand que 0 alors on fait « x--; attente = 8; } ». Sinon, on passe.
`if (bouton(BAS) && y < 17)  { y++; attente = 8; }`	Une décision : si le bouton BAS est enfoncé et y est plus petit que 17 alors on fait « y++; attente = 8; } ». Sinon, on passe.
`if (bouton(HAUT) && y > 0)  { y--; attente = 8; }`	Une décision : si le bouton HAUT est enfoncé et y est plus grand que 0 alors on fait « y--; attente = 8; } ». Sinon, on passe.
`}`	L'accolade fermante : la fin du bloc ouvert plus haut par « { ».

`poser(x, y, ALPHABET[0]);` `// la lettre à sa place`	Appel de la fonction poser de la console, avec les arguments x, y, ALPHABET[0]. Ce qu'elle fait : pose une tuile sur la grille du fond — un nom de Tuile, ou un numéro. Un 4e argument est une TAILLE : poser(2, 3, MUR, 5) dessine MUR cinq fois plus grand dans le fond (colonne et ligne en clair). (Le commentaire le dit : « la lettre à sa place ».)
`}`	L'accolade fermante : la fin du bloc ouvert plus haut par « { ».
`}`	L'accolade fermante : la fin du bloc ouvert plus haut par « { ».

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **393 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

```
Gravé dans la cartouche :
  démarrage        60 o  ← la console, à l'allumage  — la mise en route de la console
  main()          149 o  ← démarrage  — ta fonction
  VBlank            8 o  ← la console, à chaque image  — compte les images, 60 fois par seconde
  AttendreImage    28 o  ← image()  — attend l'image suivante
  AttendreVBlank   16 o  ← démarrage  — attend que l'écran soit libre
  AttendreAcces    12 o  ← EffacerCases  — attend de pouvoir écrire une case
  AdresseCase      10 o  ← effacer(), poser()  — trouve l'adresse d'une case de l'écran
  EffacerCases     12 o  ← effacer()  — efface des cases
  CopierTuiles     31 o  ← démarrage  — copie les lettres et les dessins en mémoire vidéo
  EffacerCarte     14 o  ← démarrage  — vide les deux cartes de l'écran
  LireManette      37 o  ← bouton()  — lit les huit boutons
  Tuiles           16 o  ← CopierTuiles  — 1 lettre sur 43
Lettres gravées : A
Pas gravé (jamais appelé) : AvancerAirs, AttendreFinVBlank, EcrireTexte, EcrireNombre, Diviser, Reste, Multiplier,
DecalerGauche, DecalerDroite, RangerLutins, InstallerTransfert, Hasard, EffacerFond, EffacerPanneau,
DonneesTransfert, Notes
Fonctions du langage appelées :
  bouton()  × 4    20 o sur place, dans main()  + LireManette (37 o)
  effacer() × 1    15 o sur place, dans main()  + AdresseCase (10 o), EffacerCases (12 o)
  poser()   × 1    13 o sur place, dans main()  + AdresseCase (10 o)
  image()   × 1     3 o sur place, dans main()  + AttendreImage (28 o)
```

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```
#include <effacer>    // efface des cases, ou tout le fond
#include <bouton>     // lit un bouton de la manette
#include <poser>      // pose une tuile sur une case du fond
#include <ALPHABET>   // les lettres de la police : ALPHABET[0] est le A
```

2. UNE VARIABLE — X

```
uint8_t x = 9;        // la lettre part du milieu de l'écran
```

3. UNE VARIABLE — Y

```
uint8_t y = 8;
```

4. UNE VARIABLE — ATTENTE

```
uint8_t attente = 0;  // les images à attendre avant le prochain pas
```

5. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() {
  while (true) {
    image();

    if (attente > 0) attente--;

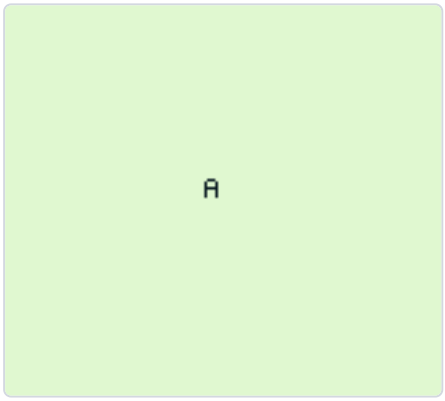
    if (attente == 0) {
      effacer(x, y, 1);                              // efface l'ancienne place
      if (bouton(DROITE) && x < 19) { x++; attente = 8; }
      if (bouton(GAUCHE) && x > 0)  { x--; attente = 8; }
      if (bouton(BAS) && y < 17)    { y++; attente = 8; }
      if (bouton(HAUT) && y > 0)    { y--; attente = 8; }
    }

    poser(x, y, ALPHABET[0]);                        // la lettre à sa place
  }
}
```

3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.

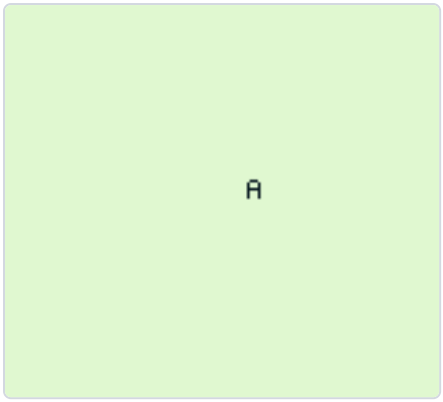
la toute première image — la console vient de démarrer — image 1



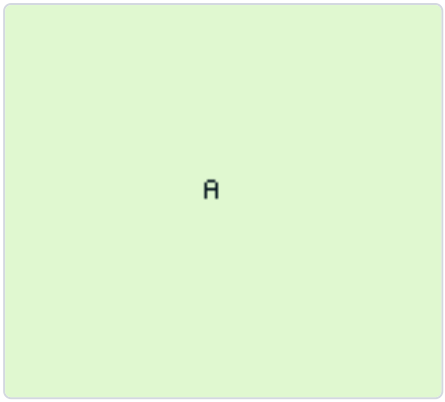
un tiers de seconde plus tard : le premier dessin est posé — image 20

4. Ce que les touches changent

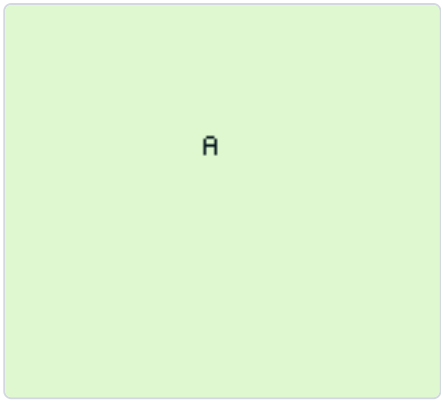
Chaque bouton que le programme lit a été enfoncé douze images, puis relâché. Voici l'écran juste après.



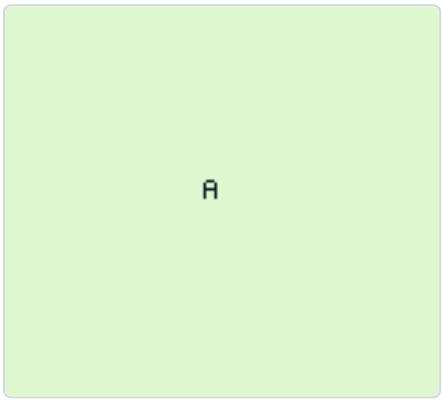
DROITE — l'écran a changé



GAUCHE — l'écran a changé



HAUT — l'écran a changé



BAS — l'écran a changé

5. Ce qu'on doit voir

> Un A au milieu de l'écran, qui se déplace avec les flèches.

6. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

- ✓ le A part du milieu
- ✓ la flèche droite le déplace
- ✓ la flèche bas aussi
- ✓ il est à sa nouvelle place
- ✓ et plus au départ

Ensuite : **0.66.1. La lettre bouge avec la croix — de base, ailleurs** — chapitre 0 : Avant tout. · gameboy3 — écrire une cartouche Game Boy en C++